

Code Sentinel

Statik Kod Analizi ve Hata Tespit Sistemi — C dilinde yazılmış kaynak kodları **çalıştırmadan analiz eden** modüler bir sistem.

Necmettin Erbakan Üniversitesi

CodeSentinel Team | BIL 204 | 2026



Ekip & Görev Dağılımı

Hüseyin Mert KANLIDERE

Altyapı & Veri Katmanı — Preprocessor, FileLoader,
DiagnosticSeverity

Mehmet UÇAR

Sözcüksel Analiz — Lexer, TokenType, R003/R004/R005

Mehmet Enes İYİŞENYÜREK

Sözdizimsel Analiz — Parser, ASTNode, R006

Hacı Ömer OĞUZ

Syntax Analizi & Kural Motoru — SyntaxAnalyzer,
RuleEngine, R001/R002/R012

Ferhat KOCABIYIK

Pointer Kuralları & Metrikler — R007–R011,
MetricCalculator

Kemal BABAOĞLAN

Raporlama & Entegrasyon — Exporters, ReportManager,
AnalysisEngine

Problem: C Dilinde Kritik Hatalar

C dilinde bellek yönetimi ve pointer kullanımı nedeniyle ciddi hatalar oluşabilmektedir. CodeSentinel bu hataları kodu çalıştırmadan tespit eder.

Variable & Code Quality Issues

R001

Kullanılmayan Değişken

R002

Başlatılmamış Değişken

R003

Magic Number

R004

Fonksiyon Uzunluğu

R005

goto Kullanımı

R006

Sonsuz Döngü

Memory Management Errors

R007

NULL Kontrolü Eksik

R008

Free Sonrası Kullanım

R009

Bellek Sızıntısı

R010

Çift Free

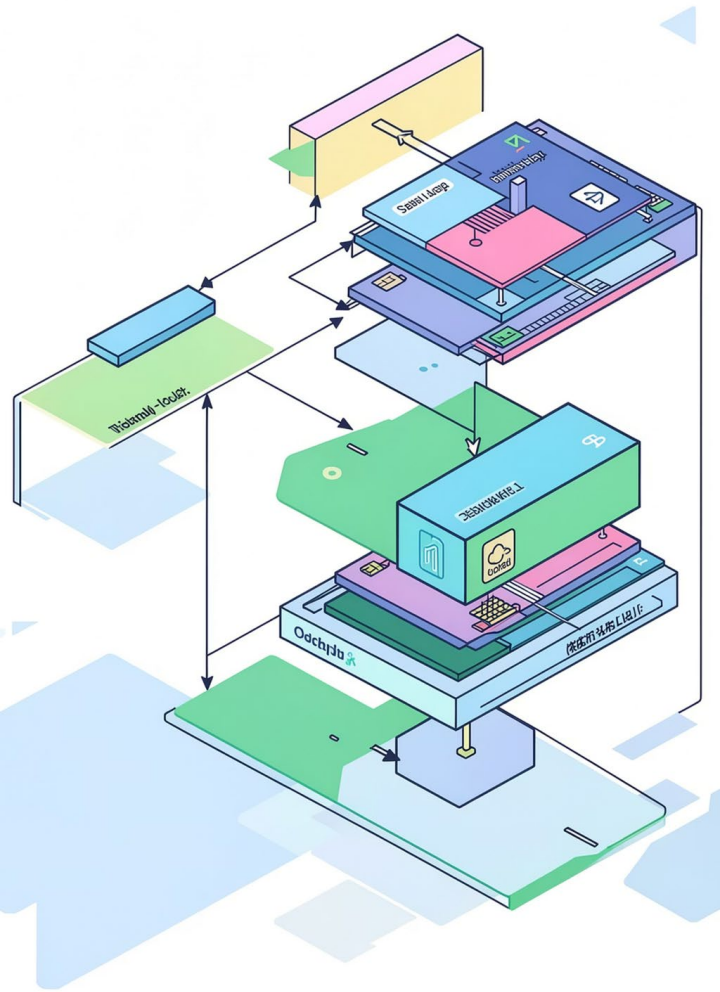
R011

Dangling Pointer

R012

Başlatılmamış Pointer





Sistem Tasarımı

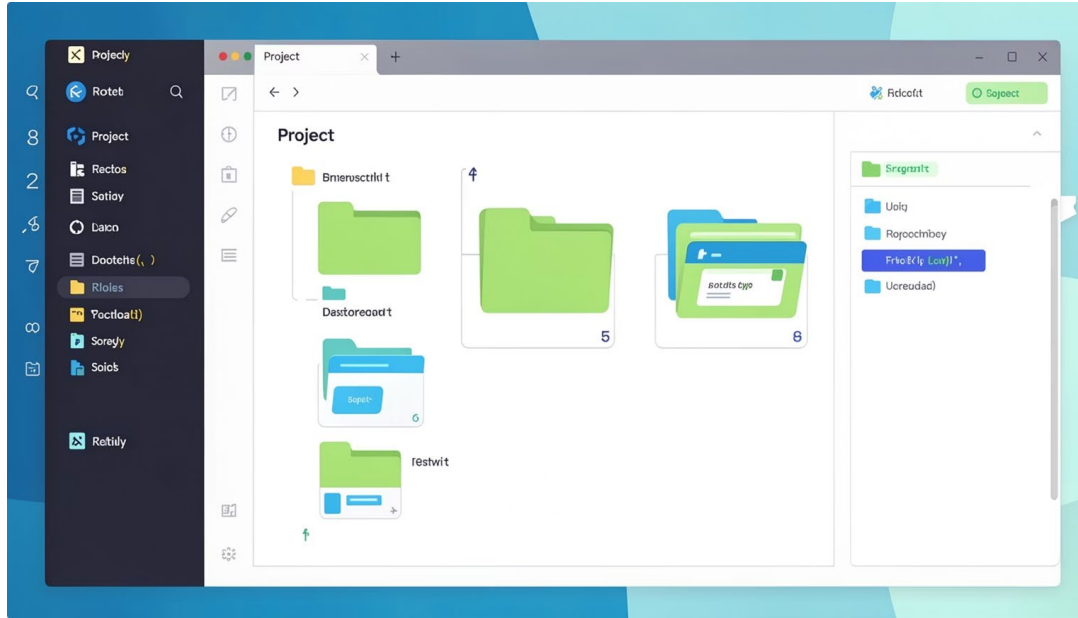
Katmanlı ve modüler mimari — her bileşen bağımsız geliştirilebilecek şekilde tasarlanmıştır.

C++ & OOP

CMake

Github

Modüler Klasör Yapısı

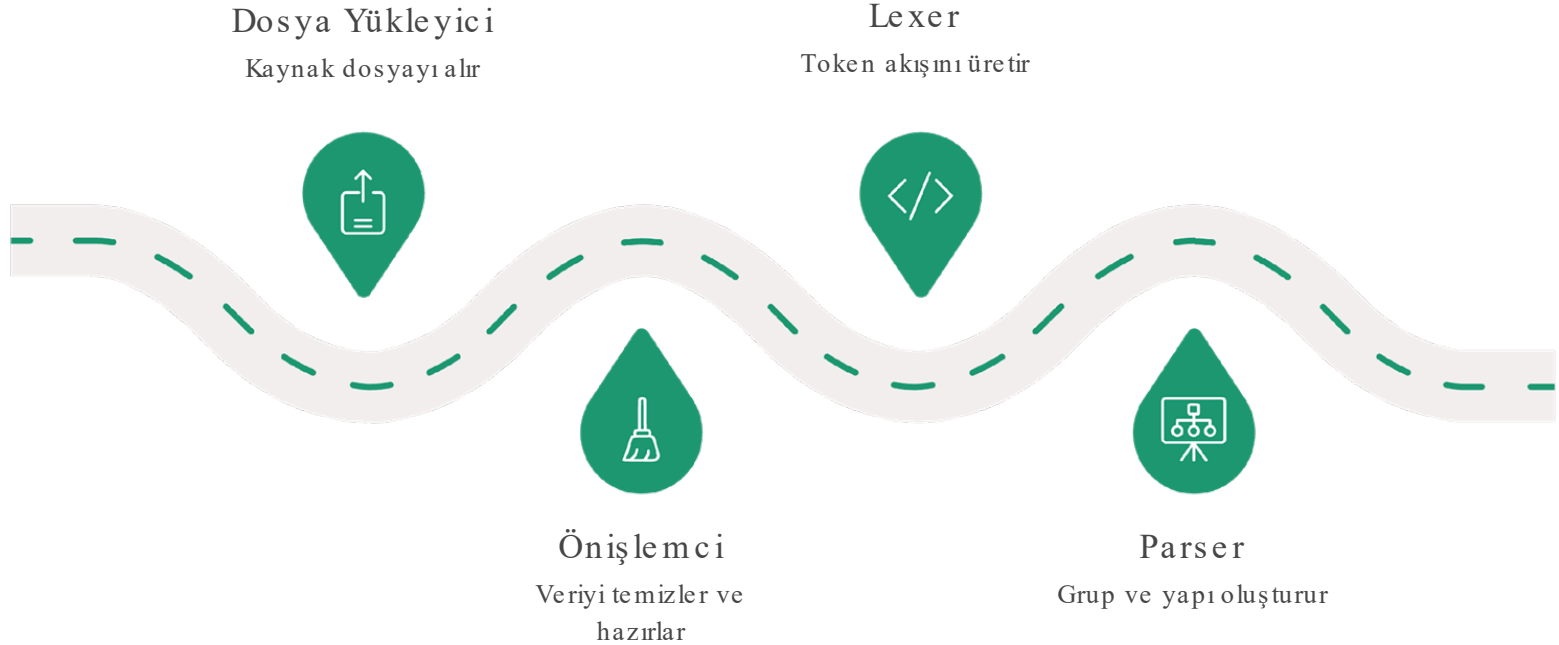


Temel Modüller

- **lexer** — Token üretimi
- **parser** — AST oluşturma
- **syntax_analyzer** — Sözdizimi kontrolü
- **analysis_engine** — Kural motoru
- **pointer** — Pointer analizi
- **infrastructure** — Altyapı servisleri

Analiz Akışı 1

Kaynak dosyadan nihai rapora uzanan dört aşamalı akış:



Her aşama bir sonrakine yapılandırılmış veri ileterek hata tespitini sistematik hale getirir.

Lexer

Bu klasörde ilk önce kod temizlenir sonrasında kod içerisinde anlama ifade eden bilgiler çıkarılır.

```
Kaynak metin:   i   n   t       y       =       2   0   ;
                  └───┬───┬───┬───┬───┘
                  ▼   ▼   ▼   ▼   ▼
Token #1: { type: KEYWORD,    value: "int",  line: 7, col: 5  }
Token #2: { type: IDENTIFIER, value: "y",   line: 7, col: 9  }
Token #3: { type: OPERATOR,   value: "=",    line: 7, col: 11 }
Token #4: { type: INT_LITERAL, value: "20",  line: 7, col: 13 }
Token #5: { type: PUNCTUATION, value: ";",   line: 7, col: 15 }
```

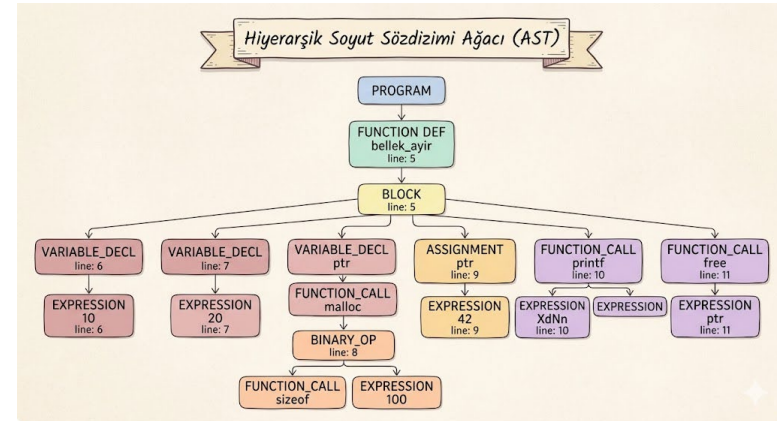
Parser

Lexer tarafından üretilen token dizisini hiyerşik bir ağaç yapısına dönüştürür.

Kaynak Kod (Örnek)

```
5 void bellek_ayir() {
6     int x = 10;
7     int y = 20;
8     int *ptr = malloc(sizeof(int) * 100);
9     ptr[0] = 42;
10    printf("%d\n", x);
11    free(ptr);
12    return 0;
13 }
```

```
[void] [bellek_ayir]
[x] [10] [int] [y]
[20] [5] [int] [*]
[ptr][-] [malloc]
[(] [sizeof][100] [)]
[ ; ] [42] [ ; ]
[printf] [(] [ "XdXn" ]
[return] [0] [ ; ]
```



SyntaxAnalyzer

Token listesi ve AST üzerinden kodun sözdizimsel analizini yapar.

Demonun bulabildiği syntax hataları aşağıdaki gibidir:

- Noktalı virgül (;) olması gereken yerlerde noktalı virgül var mı?
- Açılan parantezler - { ([]) } - kapatılmış mı?
- Tipi olmadan başlatılan değişken var mı?



RuleEngine

AST üzerinde dolaşarak koddaki derlemeyi engellemeyen ancak yazılmaması gereken komutları tespit eder. Demoda tespit edilebilen hatalar şunlardır:

- Kullanılmamış değişken.
- Pointer'lar başlatılırken NULL kontrolü yapılmış mı?



Infrastructure

Sistem altyapısının merkezinde, kod analizinden elde edilen her bir bulguyu temsil eden **Diagnostic** nesnesi yer alır.



Diagnostic Nesnesi

Hata konumu, mesaj içeriği, önem seviyesi ve ilgili kod parçasını içeren temel veri yapısıdır.

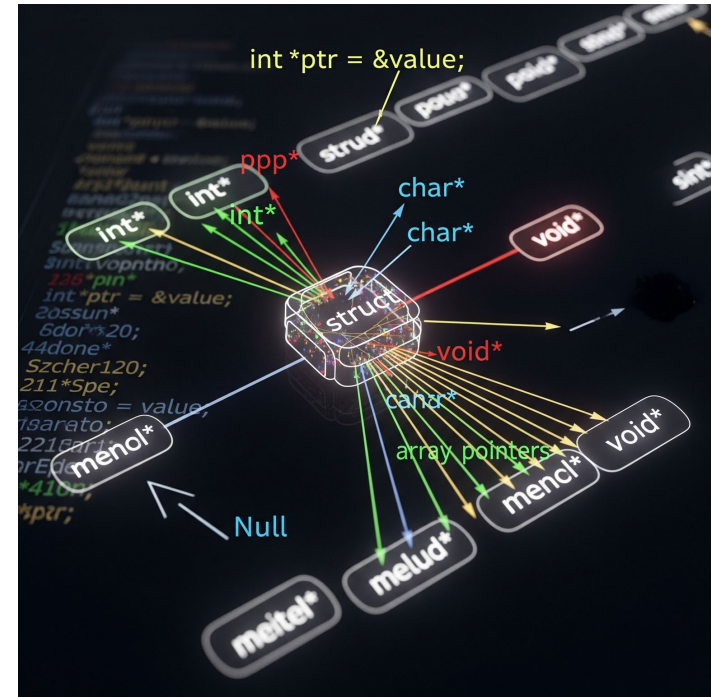
```
Diagnostic(  
  line: 7,  
  column: 5,  
  message: "Değişken 'y' kullanılmamış",  
  severity: WARNING,  
  ruleId: "R001",  
  codeSnippet: "int y = 20;"  
)
```

Pointer

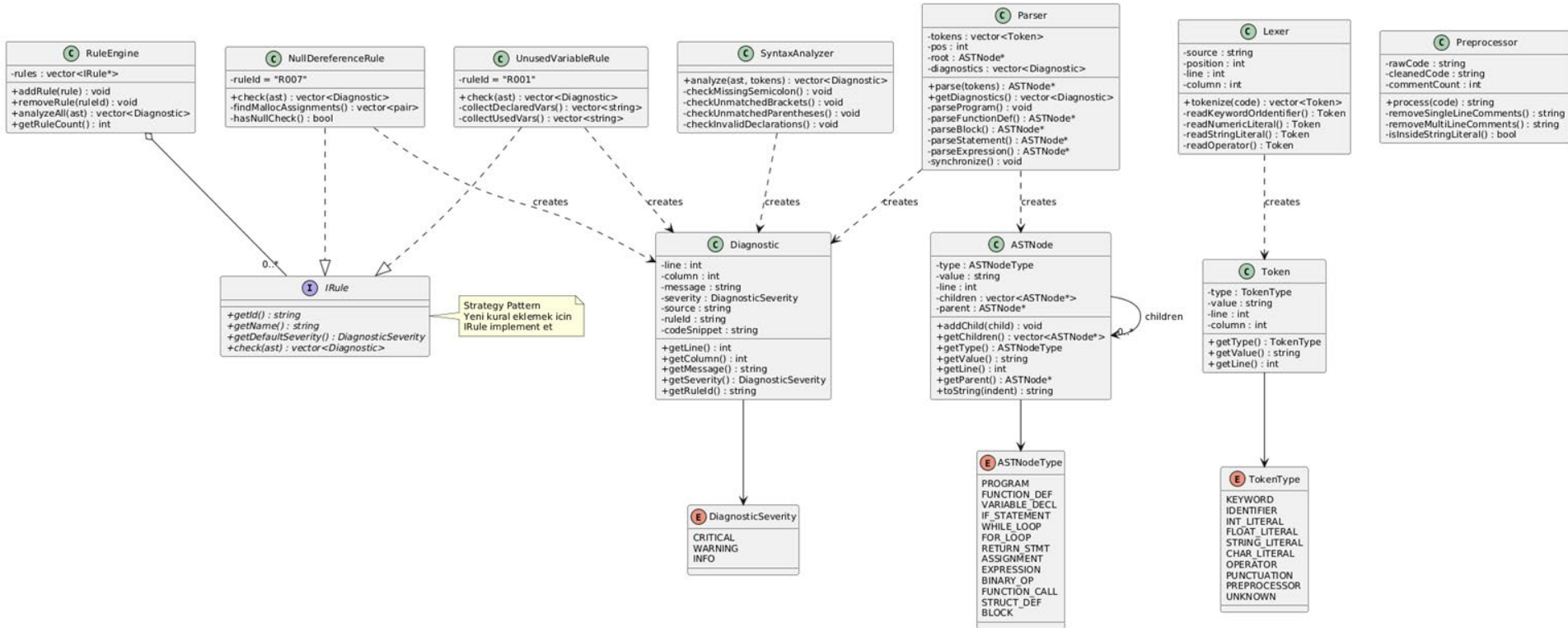
İşaretçiler belleğin rastgele bir bölgesine sabit zamanda erişebilmesi gibi özellikleri sayesinde C dilinin en önemli özelliklerinden biridir. Ancak bu hız ve esneklik yanında bellek hatalarını da getirir. Pointer klasörünün içerisindeki kodlar ile bu hatlardan bazılarını tespit etmekteyiz.

Tespit edilebilen pointer hataları:

- Free sonrası kullanım - CRITICAL
- Çift Free - CRITICAL
- Bellek sızıntısı - WARNING
- Dangling pointer - INFO



Class Diagram



Canlı Demo: Tespit Edilen Hatalar

Unused Variable Rule

Tanımlanmış ancak hiç kullanılmayan değişkenler

Uninitialized Var Rule

Değer atanmadan kullanılan değişkenler

Eksik NULL Kontrolü

malloc sonrası NULL kontrol edilmeden kullanım

Free Sonrası Kullanım

free() sonrası pointer erişimi

Başlatılmamış Pointer

Değer atanmadan dereference edilen pointer

Canlı Demo

Adım adım gerçek zamanlı analiz gösterimi:

01 – Girdi Hazırlığı

.c dosyası yüklenir ve ön işlem yapılır.

02 – Lexer & Parser

Kod token'lara ayrılır ve AST oluşturulur.

03 – Rule Engine Analizi

Statik analiz kuralları çalıştırılır.

04 – Çıktılar

Hatalar HTML dosyası şeklinde gösterilir.





Karşılaşılan Zorluklar

Parser Hata Kurtarması

Parser hata bulduğunda durmak yerine diagnostics listesine ekleyip bir sonraki `;` veya `}`'e kadar atlayarak kısmi AST üretiyor. Tek seferde birden fazla hata raporlanabiliyor.

String İçindeki Yorum Karakterleri (Ön İşleme)

`char *s = "// bu yorum değil"` gibi durumlarda `isInsideStringLiteral()` fonksiyonu ile string içi `//` karakterleri korunuyor.

Ekip Koordinasyonu

Entegrasyonda arayüz uyumsuzlukları çıktı. Çözüm: tüm sınıf arayüzleri ve metod imzaları ortak dokümanda tanımlandı; herkes bu sözleşmeye uydu.

Proje Durumu

✓ Tamamlananlar

- C kaynak kodu okuma & ön işleme
- Parser hata kurtarma mekanizması
- R001, R002, R007, R008 ve R012 ID'li kurallar.
- Modüler kural yapısı (Strategy Pattern)
- Birden fazla .c dosyasını tarayıp birleşik proje raporu üretme desteği
- HTML rapor verme
(● kritik, ● uyarı, ● bilgi)

ID	Kural Adı	Sınıf	Seviye
R001	Kullanılmayan Değişken	UnusedVariableRule	WARNING
R002	Başlatılmamış Değişken	UninitializedVarRule	WARNING
R003	Magic Number	MagicNumberRule	WARNING
R004	Fonksiyon Uzunluğu	FunctionLengthRule	WARNING
R005	goto Kullanımı	GotoUsageRule	WARNING
R006	Sonsuz Döngü	InfiniteLoopRule	WARNING
R007	NULL Kontrolü Eksik	NullDereferenceRule	WARNING
R008	Free Sonrası Kullanım	UseAfterFreeRule	CRITICAL
R009	Bellek Sızıntısı	MemoryLeakRule	WARNING
R010	Çift Free	DoubleFreeRule	CRITICAL
R011	Dangling Pointer	DanglingPointerRule	INFO
R012	Başlatılmamış Pointer	UninitializedPointerRule	CRITICAL

Gelecek Çalışmalar

Statik Kod Analizi ve Hata Tespit Sistemi, C kaynak kodlarını statik olarak analiz eden, OOP prensipleriyle tasarlanmış, **genişletilebilir** bir araçtır.



Kalan 7 Kural

R003 , R004 , R005 , R006 , R009 , R010 ve R011

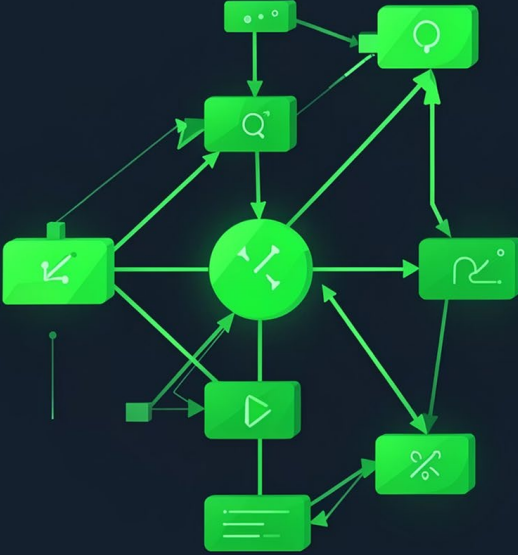


GUI Arayüz

Kullanıcı dostu grafik arayüz geliştirme

Metrik Hesaplama

LOC , Kod Satırı , Değişken Sayısı , Fonksiyon Sayısı gibi



Geliřtirme Hedefleri

IDE Entegrasyonu

Popüler geliřtirme ortamlarına entegrasyon

Otomatik Fix Önerileri

Bazı hatalar için çözüm önerisi sunma

Daha Fazla C Standardı

Geniřletilmiş kural seti ve standart desteęi





Dinlediğiniz İçin Teşekkürler

Proje Bilgileri

- Ders: BİL 204 /
YAZILIM MÜHENDİSLİĞİ
- Bölüm: BİLGİSAYAR MÜHENDİSLİĞİ Hüseyin Mert KANLIDERE
- Grup: CodeSentinel
- Tarih: 11.05.2026

Takım Üyeleri

- Ferhat Enes KOCABIYIK
- Hacı Ömer OĞUZ
- Hüseyin Mert KANLIDERE
- Kemal BABAOĞLAN
- Mehmet Enes İYİŞENYÜREK
- Mehmet UÇAR